

Lab B: convolution-tutorial

Ching Wen, Chen

Contents

- Performance Estimation
- Learn from Kernel Code Implementation
- Experimental Result
- Future Work

Performance Estimation

A 2D convolution filter (e.g. a 3×3 smoothing or edge-detection kernel) is at the heart of many video-processing pipelines (noise reduction, color enhancement, motion blur, etc.). Before accelerating this filter on FPGA, we must:

1. **Establish a software baseline** by measuring the filter's performance on the host CPU
2. **Translate real-world video requirements** (HD-1080p at 60 FPS) into a pixel-throughput target
3. **Estimate the hardware acceleration** needed to meet or exceed that target

Performance Estimation

Key Metrics

- **Resolution:** 1920×1080 pixels/frame
- **Frame rate:** 60 frames/sec

Total pixels per second can be written step by step as:

- Pixels per frame:
 $1920 \times 1080 = 2\,073\,600$ pixels/frame
- Frames per second: 60
- Therefore,

$$2\,073\,600 \text{ pixels/frame} \times 60 \text{ frames/s} = 124\,416\,000 \text{ pixels/s} \approx 124.4 \times 10^6 \text{ pixels/s}$$

So the system must process about **124.4 million pixels every second**.

Performance Estimation

Acceleration Factor

$$\text{Accel} = \frac{\text{Target throughput}}{\text{Software throughput}} = \frac{124.4 \text{ MP/s}}{8.3 \text{ MP/s}} \approx 15\times$$

This tells us our FPGA design must be at least 15× faster than the CPU version

Implementation Plan

HLS Kernel

II = 1, fully pipelined loops, array partitioning for coefficients, DATAFLOW across modules

Multi-Bank Memory

Split AXI bundles across DDR/HBM channels to avoid bottleneck

Compute Scaling

Start with 1 CU; verify 250 MP/s; then replicate to 2 CUs to prove ~500 MP/s

Host Overlap

Use asynchronous data transfers (double-buffering) to hide PCIe latency

Outlines for Kernel Code

- ReadFromMem Module
- Window2D Module
- Filter2D Module
- WriteToMem Module
- HLS Optimization Directives
- Memory & Burst Optimization
- Streaming Architecture

ReadFromMem Module

- Auto-widened AXI4-MM reads from src and coeffs
- Padding coefficients to multiple-of-64 for burst efficiency
- Streaming coeffs and pixel data into HLS streams

Window2D Module

- Line buffers store FILTER_V_SIZE-1 previous rows
- Sliding window of size FILTER_V_SIZE×FILTER_H_SIZE
- Pipelined loop with II=1 for continuous window generation

```
void Window2D(
    unsigned short    width,
    unsigned short    height,
    hls::stream<U8>    &pixel_stream,
    hls::stream<window> &window_stream)
{
    // Line buffers - used to store [FILTER_V_SIZE-1] entire lines of pixels
    U8 LineBuffer[FILTER_V_SIZE-1][MAX_IMAGE_WIDTH];
    #pragma HLS ARRAY_PARTITION variable=LineBuffer dim=1 complete
    #pragma HLS DEPENDENCE variable=LineBuffer inter false
    #pragma HLS DEPENDENCE variable=LineBuffer intra false

    // Sliding window of [FILTER_V_SIZE][FILTER_H_SIZE] pixels
    window Window;

    unsigned col_ptr = 0;
    unsigned ramp_up = width*((FILTER_V_SIZE-1)/2)+(FILTER_H_SIZE-1)/2;
    unsigned num_pixels = width*height;
    unsigned num_iterations = num_pixels + ramp_up;

    const unsigned max_iterations = MAX_IMAGE_WIDTH*MAX_IMAGE_HEIGHT + MAX_IMAGE_WIDTH*((FILTER_V_SIZE-1)/2);
}
```

Window2D Module

- Line buffers store $\text{FILTER_V_SIZE}-1$ previous rows
- Sliding window of size $\text{FILTER_V_SIZE} \times \text{FILTER_H_SIZE}$
- Pipelined loop with $\text{II}=1$ for continuous window generation

```
// Iterate until all pixels have been processed
update_window: for (int n=0; n<num_iterations; n++)
{
#pragma HLS LOOP_TRIPCOUNT max=max_iterations
#pragma HLS PIPELINE II=1

    // Read a new pixel from the input stream
    U8 new_pixel = (n<num_pixels) ? pixel_stream.read() : 0;

    // Shift the window and add a column of new pixels from the line buffer
    for(int i = 0; i < FILTER_V_SIZE; i++) {
        for(int j = 0; j < FILTER_H_SIZE-1; j++) {
            Window.pix[i][j] = Window.pix[i][j+1];
        }
        Window.pix[i][FILTER_H_SIZE-1] = (i<FILTER_V_SIZE-1) ? LineBuffer[i][col_ptr] : new_pixel;
    }

    // Shift pixels in the column of pixels in the line buffer, add the newest pixel
    for(int i = 0; i < FILTER_V_SIZE-2; i++) {
        LineBuffer[i][col_ptr] = LineBuffer[i+1][col_ptr];
    }
    LineBuffer[FILTER_V_SIZE-2][col_ptr] = new_pixel;

    // Update the line buffer column pointer
    if (col_ptr==(width-1)) {
```

Window2D Module

- burst length: 16 (default)

```
#pragma HLS INTERFACE m_axi \  
    port=dst \  
    bundle=mem1 \  
    ... \  
    max_write_burst_length=64 \  
    num_write_outstanding=8
```

```
#pragma HLS INTERFACE m_axi \  
    port=src \  
    bundle=mem0 \  
    offset=slave \  
    depth=... \  
    max_read_burst_length=64 \  
    num_read_outstanding=8
```

Filter2D Module

- Local coefficient storage with
ARRAY_PARTITION
- Nested loops pipelined at II=1 for per-pixel
compute
- Normalization and clamping using factor and
bias

```
void Filter2D(
    unsigned short    width,
    unsigned short    height,
    float             factor,
    short             bias,
    hls::stream<char> &coeff_stream,
    hls::stream<window> &window_stream,
    hls::stream<U8>    &pixel_stream )
{
    assert(width  <= MAX_IMAGE_WIDTH);
    assert(height <= MAX_IMAGE_HEIGHT);

    // Filtering coefficients
    char coeffs[FILTER_V_SIZE][FILTER_H_SIZE];
    #pragma HLS ARRAY_PARTITION variable=coeffs complete dim=0

    // Load the coefficients into local storage
    load_coefs: for (int i=0; i<FILTER_V_SIZE; i++) {
        for (int j=0; j<FILTER_H_SIZE; j++) {
            #pragma HLS PIPELINE II=1
            coeffs[i][j] = coeff_stream.read();
        }
    }
}
```

Filter2D Module

- Local coefficient storage with
ARRAY_PARTITION
- Nested loops pipelined at II=1 for per-pixel
compute
- Normalization and clamping using factor and
bias

```
void Filter2D(
    unsigned short    width,
    unsigned short    height,
    float             factor,
    short             bias,
    hls::stream<char> &coeff_stream,
    hls::stream<window> &window_stream,
    hls::stream<U8>    &pixel_stream )
{
    assert(width  <= MAX_IMAGE_WIDTH);
    assert(height <= MAX_IMAGE_HEIGHT);

    // Filtering coefficients
    char coeffs[FILTER_V_SIZE][FILTER_H_SIZE];
    #pragma HLS ARRAY_PARTITION variable=coeffs complete dim=0

    // Load the coefficients into local storage
    load_coefs: for (int i=0; i<FILTER_V_SIZE; i++) {
        for (int j=0; j<FILTER_H_SIZE; j++) {
            #pragma HLS PIPELINE II=1
            coeffs[i][j] = coeff_stream.read();
        }
    }
}
```

WriteToMem Module

- Streaming output pixels back to global memory
- Burst writes aligned to 64 bytes for bandwidth
- Zero padding ensures stride alignment

```
}  
  
// Process the incoming stream of pixel windows  
apply_filter: for (int y = 0; y < height; y++)  
{  
    for (int x = 0; x < width; x++)  
    {  
#pragma HLS PIPELINE II=1  
        // Read a 2D window of pixels  
        window w = window_stream.read();  
  
        // Apply filter to the 2D window  
        int sum = 0;  
        for(int row=0; row<FILTER_V_SIZE; row++)  
        {  
            for(int col=0; col<FILTER_H_SIZE; col++)  
            {  
                unsigned char pixel;  
                int xoffset = (x+col-(FILTER_H_SIZE/2));  
                int yoffset = (y+row-(FILTER_V_SIZE/2));  
                // Deal with boundary conditions :  
                // Clamp pixels to 0 when outside of image  
                if ( (xoffset<0) || (xoffset>=width)  
                    || (yoffset<0) || (yoffset>=height) ) {  
                    pixel = 0;  
                } else {  
                    pixel = w.pix[row][col];  
                }  
                sum += pixel*(char)coeffs[row][col];  
            }  
        }  
  
        // Normalize result  
        unsigned char outpix = MIN(MAX((int)(factor * sum)+bias), 0), 255);  
  
        // Write the output pixel  
        pixel_stream.write(outpix);  
    }  
}
```

HLS Optimization Directives

- `#pragma HLS DATAFLOW` for concurrent module execution
- `#pragma HLS PIPELINE` to achieve $II=1$ in loops
- `#pragma HLS ARRAY_PARTITION` for parallel coef access
- `#pragma HLS DEPENDENCE` to prevent RAW hazards

Memory & Burst Optimization

- Stride padded to 64 bytes to enable auto-widening
- Ensures AXI bursts transfer large contiguous blocks
- Maximizes DDR/HBM bandwidth utilization

Streaming Architecture

- Decoupled modules connected via `hls::stream`
- FIFO depths tuned to balance producer/consumer rates
- Enables fine-grained parallelism and low latency

Experiment Result – Performance Estimates from HLS syn

- \$ vitis_hls -f build.tcl
 - Generate csyn repor for both t

```
=====
== Vitis HLS Report for 'Filter2DKernel'
=====
* Date:          Wed May 14 01:16:08 2025

* Version:       2022.2 (Build 3670227 on Oct 13 2022)
* Project:       conv_filter_prj
* Solution:      solution (Vitis Kernel Flow Target)
* Product family: virtexuplus
* Target device: xcu200-fsgd2104-2-e

=====
== Performance Estimates
=====
+ Timing:
  * Summary:
    +-----+-----+-----+-----+
    | Clock | Target | Estimated| Uncertainty|
    +-----+-----+-----+-----+
    | ap_clk | 3.33 ns| 2.433 ns| 0.90 ns|
    +-----+-----+-----+-----+

+ Latency:
  * Summary:
    +-----+-----+-----+-----+-----+-----+
    | Latency (cycles) | Latency (absolute) | Interval | Pipeline |
    | min | max | min | max | min | max | Type |
    +-----+-----+-----+-----+-----+-----+
    | 405| 2211165| 1.350 us| 7.370 ms| 406| 2211166| dataflow|
    +-----+-----+-----+-----+-----+-----+
```

```
=====
== Vitis HLS Report for 'Filter2DKernel'
=====
* Date:          Wed May 14 14:38:02 2025

* Version:       2022.2 (Build 3670227 on Oct 13 2022)
* Project:       conv_filter_prj
* Solution:      solution (Vitis Kernel Flow Target)
* Product family: virtexuplus
* Target device: xcu280-fsvh2892-2L-e

=====
== Performance Estimates
=====
+ Timing:
  * Summary:
    +-----+-----+-----+-----+
    | Clock | Target | Estimated| Uncertainty|
    +-----+-----+-----+-----+
    | ap_clk | 3.60 ns| 2.628 ns| 0.97 ns|
    +-----+-----+-----+-----+

+ Latency:
  * Summary:
    +-----+-----+-----+-----+-----+-----+
    | Latency (cycles) | Latency (absolute) | Interval | Pipeline |
    | min | max | min | max | min | max | Type |
    +-----+-----+-----+-----+-----+-----+
    | 405| 2211165| 1.458 us| 7.960 ms| 406| 2211166| dataflow|
    +-----+-----+-----+-----+-----+-----+
```

Experiment Result – Latency for core kernel and instances

```
+ Latency:
  * Summary:
+-----+-----+-----+-----+-----+-----+
| Latency (cycles) | Latency (absolute) | Interval | Pipeline |
| min | max | min | max | min | max | Type |
+-----+-----+-----+-----+-----+-----+
| 405 | 2211165 | 1.350 us | 7.370 ms | 406 | 2211166 | dataflow |
+-----+-----+-----+-----+-----+-----+
```

+ Detail:							
* Instance:							
+-----+-----+-----+-----+-----+-----+-----+-----+							
		+-----+-----+		+-----+-----+		+-----+-----+	
Module	Interval	Instance	Pipeline Module	Latency (cycles)		Latency (abs)	
				min	max	min	max
max	min	max	Type				
+-----+-----+-----+-----+-----+-----+-----+-----+							
		+-----+-----+		+-----+-----+		+-----+-----+	
0 ns	entry_proc_U0	entry_proc	no	0	0	0 ns	
.370 ms	ReadFromMem_U0	ReadFromMem	no	405	2211165	1.350 us	
.956 ms	Window2D_U0	Window2D	no	8	2087055	26.664 ns	
.912 ms	Filter2D_U0	Filter2D	no	235	2073859	0.783 us	
.369 ms	WriteToMem_U0	WriteToMem	no	77	2210837	0.257 us	

```
+ Latency:
  * Summary:
  +-----+-----+-----+-----+-----+-----+
  | Latency (cycles) | Latency (absolute) | Interval | Pipeli |
  | min | max | min | max | min | max | Type |
  +-----+-----+-----+-----+-----+-----+
  | 405 | 2211165 | 1.458 us | 7.960 ms | 406 | 2211166 | dataf |
  +-----+-----+-----+-----+-----+-----+
```

+ Detail:						
* Instance:						
+-----+-----+-----+-----+-----+-----+-----+						
Module	Interval		Pipeline	Latency (cycles)		Latency
	min	max	Module	min	max	min
+-----+-----+-----+-----+-----+-----+-----+						
entry_proc_U0	0	0	entry_proc_U0	0	0	0 ns
ReadFromMem_U0	405	2211165	ReadFromMem_U0	405	2211165	1.458 us
Window2D_U0	8	2087055	Window2D_U0	8	2087055	28.800 ns
Filter2D_U0	235	2073858	Filter2D_U0	235	2073858	0.846 us
WriteToMem_U0	77	2210837	WriteToMem_U0	77	2210837	0.277 us
+-----+-----+-----+-----+-----+-----+-----+						

Experiment Result – Utilization Estimation

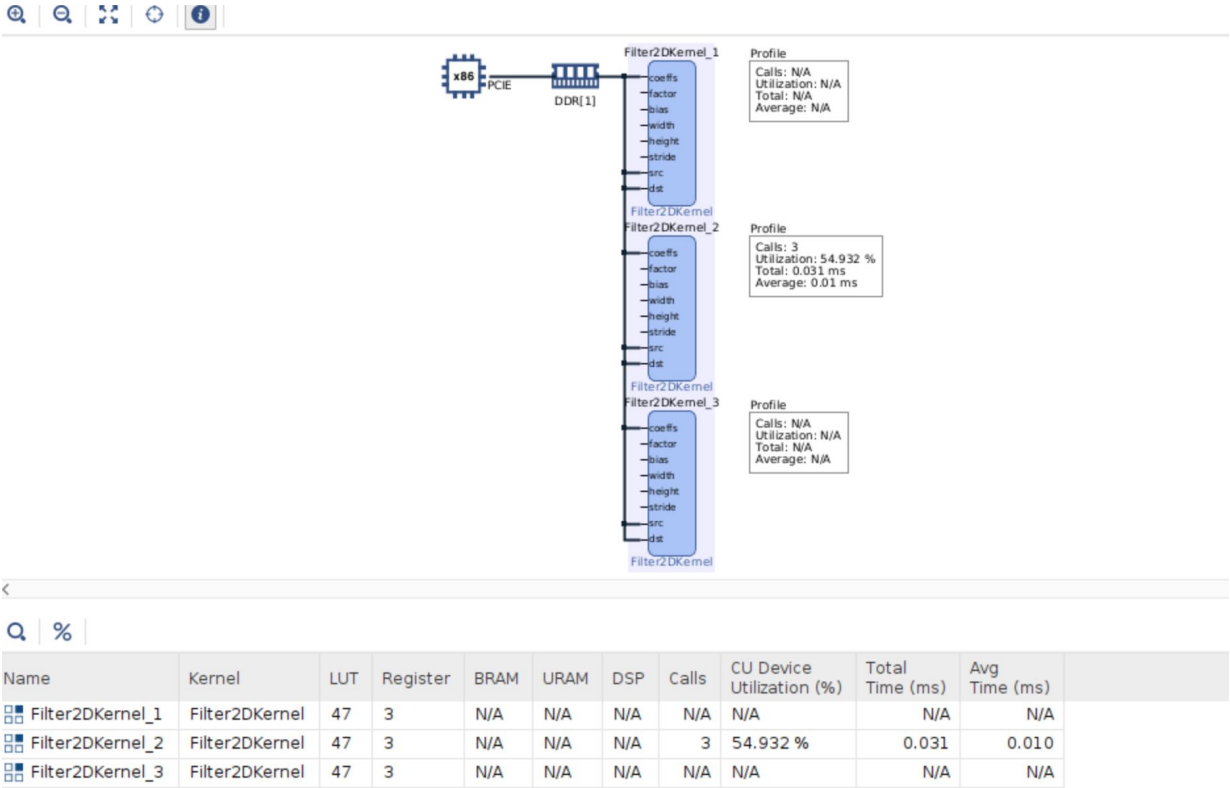
```
=====
== Utilization Estimates
=====
* Summary:
+-----+-----+-----+-----+-----+-----+
| Name | BRAM_18K | DSP | FF | LUT | URAM |
+-----+-----+-----+-----+-----+-----+
| DSP | - | - | - | - | - |
| Expression | - | - | 0 | 14 | - |
| FIFO | 0 | - | 3377 | 1944 | - |
| Instance | 14 | 139 | 26876 | 26235 | 0 |
| Memory | - | - | - | - | - |
| Multiplexer | - | - | - | 18 | - |
| Register | - | - | 5 | - | - |
+-----+-----+-----+-----+-----+-----+
| Total | 14 | 139 | 30258 | 28211 | 0 |
+-----+-----+-----+-----+-----+-----+
| Available SLR | 1440 | 2280 | 788160 | 394080 | 320 |
+-----+-----+-----+-----+-----+-----+
| Utilization SLR (%) | ~0 | 6 | 3 | 7 | 0 |
+-----+-----+-----+-----+-----+-----+
| Available | 4320 | 6840 | 2364480 | 1182240 | 960 |
+-----+-----+-----+-----+-----+-----+
| Utilization (%) | ~0 | 2 | 1 | 2 | 0 |
+-----+-----+-----+-----+-----+-----+
+ Detail:
+ * Instance:
+-----+-----+-----+-----+-----+-----+
--+
```

```
=====
== Utilization Estimates
=====
* Summary:
+-----+-----+-----+-----+-----+-----+
| Name | BRAM_18K | DSP | FF | LUT | URAM |
+-----+-----+-----+-----+-----+-----+
| DSP | - | - | - | - | - |
| Expression | - | - | 0 | 14 | - |
| FIFO | 0 | - | 3377 | 1944 | - |
| Instance | 14 | 139 | 20658 | 23259 | 0 |
| Memory | - | - | - | - | - |
| Multiplexer | - | - | - | 18 | - |
| Register | - | - | 5 | - | - |
+-----+-----+-----+-----+-----+-----+
| Total | 14 | 139 | 24040 | 25235 | 0 |
+-----+-----+-----+-----+-----+-----+
| Available SLR | 1344 | 3008 | 869120 | 434560 | 320 |
+-----+-----+-----+-----+-----+-----+
| Utilization SLR (%) | 1 | 4 | 2 | 5 | 0 |
+-----+-----+-----+-----+-----+-----+
| Available | 4032 | 9024 | 2607360 | 1303680 | 960 |
+-----+-----+-----+-----+-----+-----+
| Utilization (%) | ~0 | 1 | ~0 | 1 | 0 |
+-----+-----+-----+-----+-----+-----+
+ Detail:
+ * Instance:
+-----+-----+-----+-----+-----+-----+
--+
```

Experiment Result – System Diagram

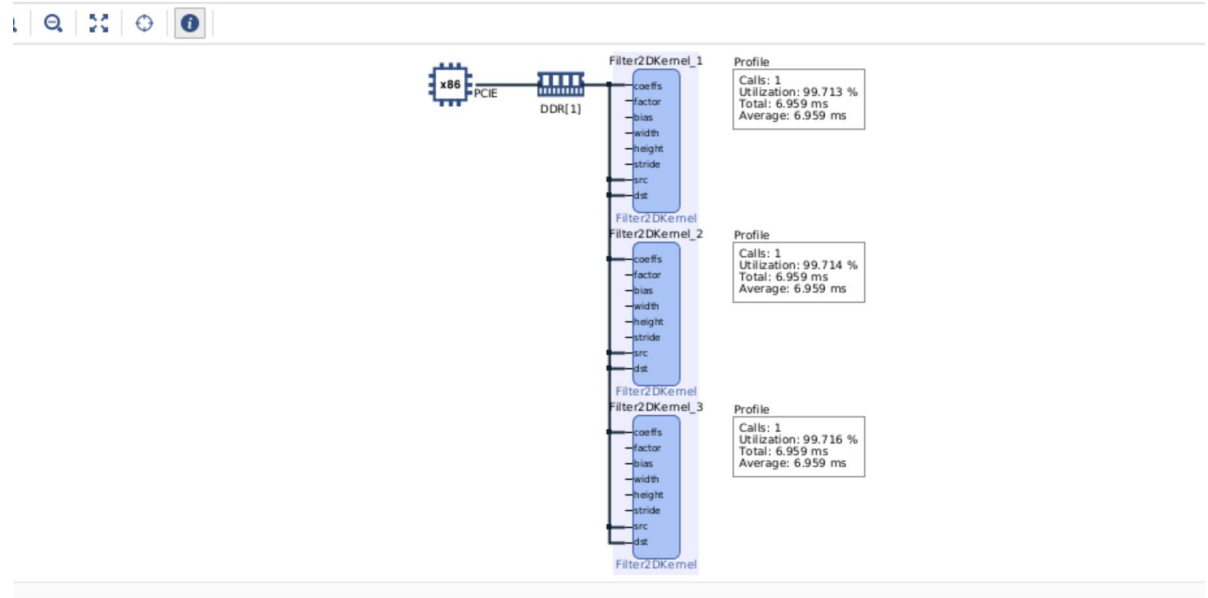
- u280 with DDR[1]
- image size:

32x32



Experiment Result – System Diagram

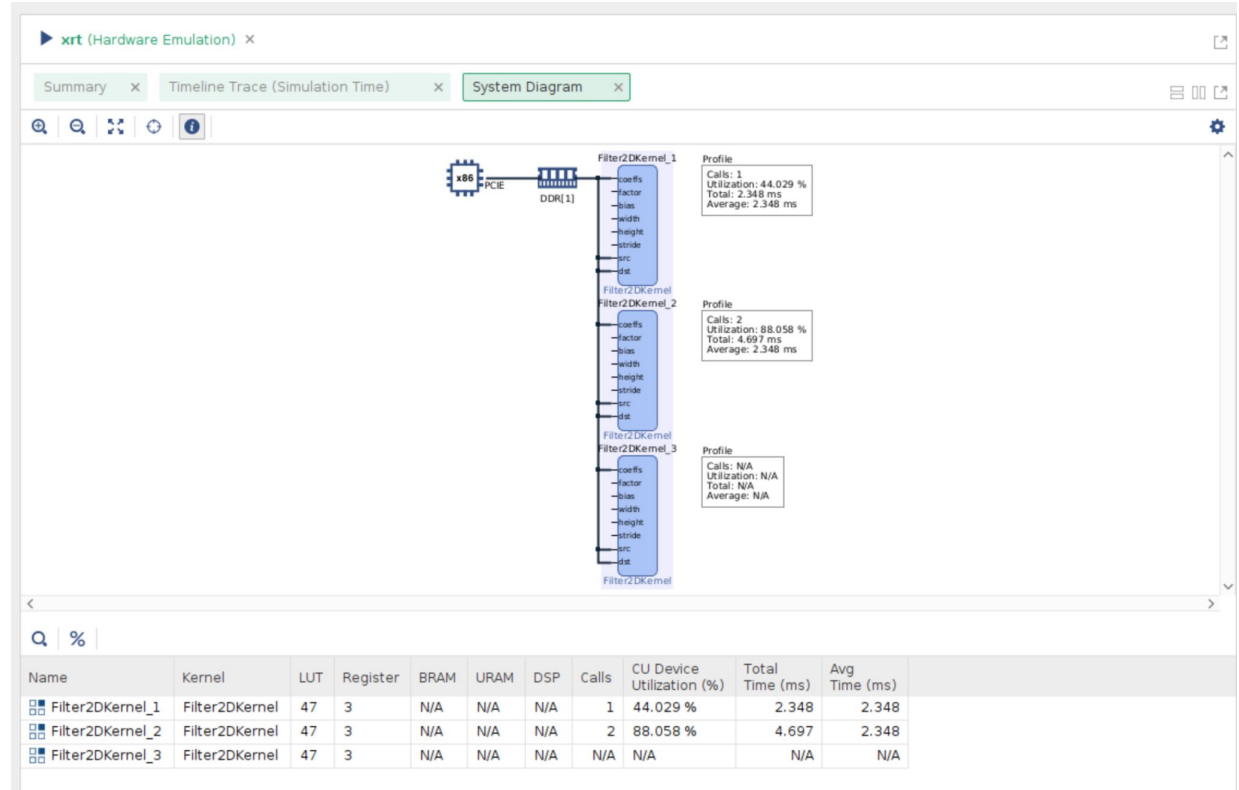
- u280 with DDR[1]
- image size:
1920x1080



ame	Kernel	LUT	Register	BRAM	URAM	DSP	Calls	CU Device Utilization (%)	Total Time (ms)	Avg Time (ms)
Filter2DKernel_1	Filter2DKernel	47	3	N/A	N/A	N/A	1	99.713 %	6.959	6.959
Filter2DKernel_2	Filter2DKernel	47	3	N/A	N/A	N/A	1	99.714 %	6.959	6.959
Filter2DKernel_3	Filter2DKernel	47	3	N/A	N/A	N/A	1	99.716 %	6.959	6.959

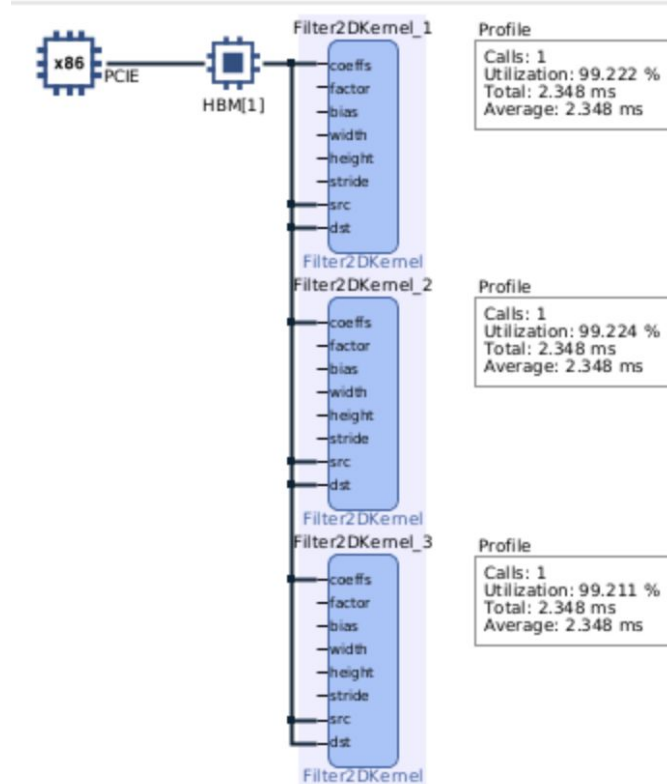
Experiment Result – System Diagram

- u280 with DDR[1]
- image size:
1080x640



Experiment Result – System Diagram

- u280 with HBM[1]
- image size:
1080x640



Experiment Result – Run Guidance

- u280 with HBM[1]
- image size:

1080x640

Name	Threshold	Actual	Details	Resolution
▼ Profiling (3)				
▼ Kernel: Filter2DKernel (1)				
▼ Argument: coeffs (1)				
KERNEL_BUFFER_PLRAM_USAGE	> 128KB	225	Buffer for argument coeffs on kernel Filter2DKernel did not use PLRAMs.	Use PLRAMs for small- to medium-sized buffers. Click here .
▼ System (2)				
▼ Device: xilinx_u280_gen3x16_xdma_1_202211_1-0 (2)				
MEMORY_READ_TRANSFER_UTIL	>= 5.000	0.000	Memory HBM[1] read utilization was 0.000% on device xilinx_u280_gen3x16_xdma_1_202211_1-0.	Improve kernel memory read efficiency to memories. Click here .
MEMORY_WRITE_TRANSFER_UTIL	>= 5.000	0.000	Memory HBM[1] write utilization was 0.000% on device xilinx_u280_gen3x16_xdma_1_202211_1-0.	Improve kernel memory write efficiency to memories. Click here .

Experiment Result – Timeline Trace

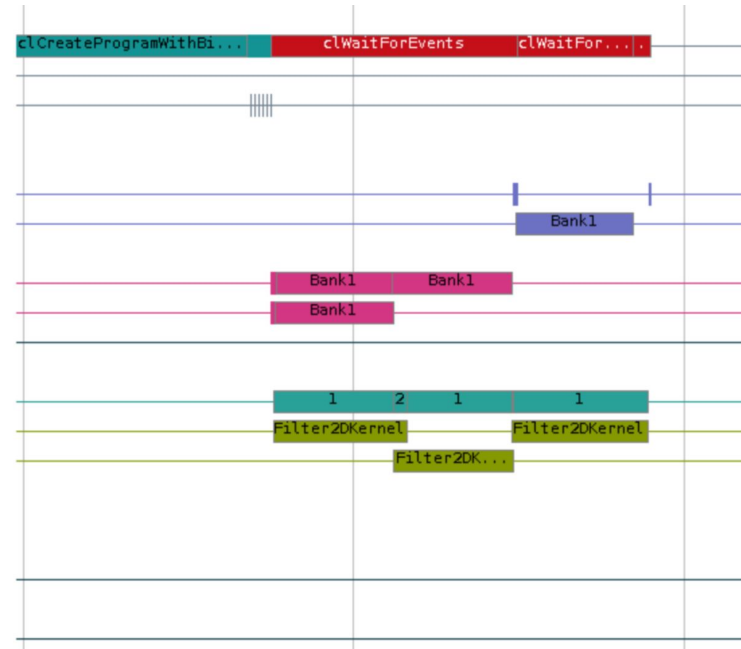
- u280 with DDR[1]
- image size:

32x32

```

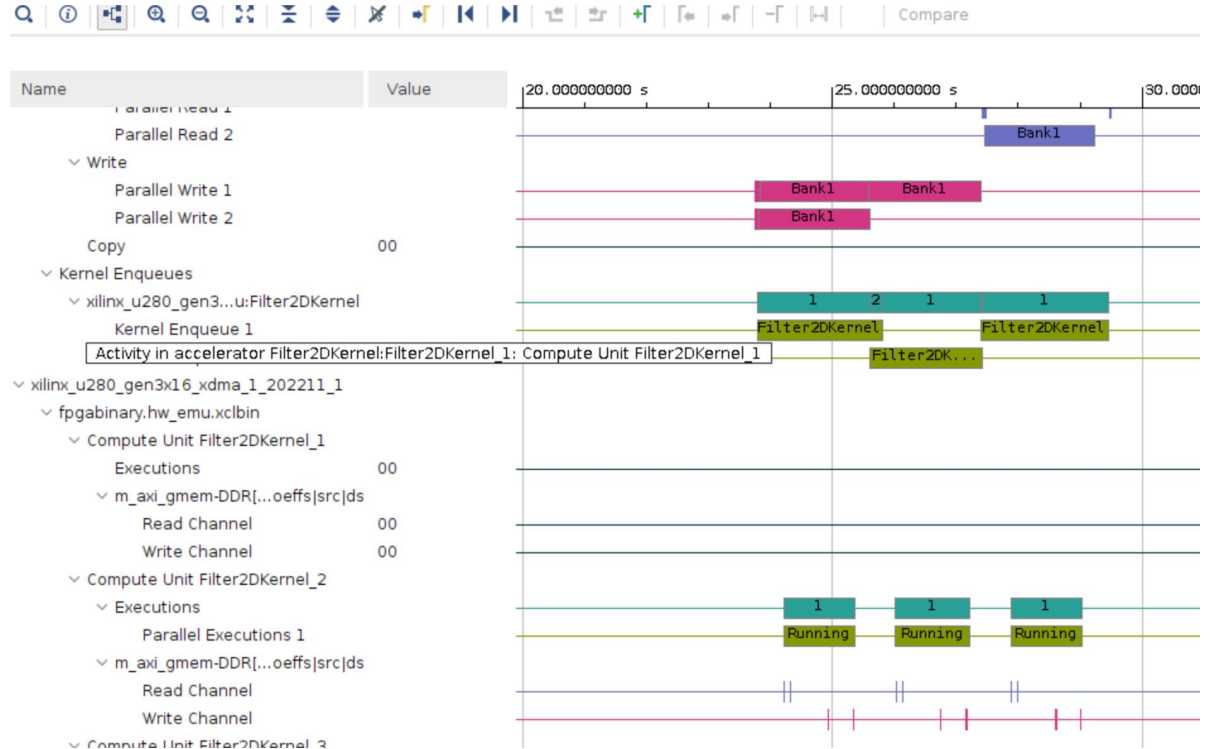
  ▾ OpenCL API Calls
    Parallel General 1
    Parallel General 2
    Queue 0x5629e0c11970
  ▾ Data Transfer
    ▾ Read
      Parallel Read 1
      Parallel Read 2
    ▾ Write
      Parallel Write 1
      Parallel Write 2
    Copy 00
  ▾ Kernel Enqueues
    ▾ xilinx_u280_gen3...u:Filter2DKernel
      Kernel Enqueue 1
      Kernel Enqueue 2
  ▾ xilinx_u280_gen3x16_xdma_1_202211_1
    ▾ fpgabinrary.hw_emu.xclbin
      ▾ Compute Unit Filter2DKernel_1
        Executions 00
      ▾ m_axi_gmem-DDR[...oeffs|src|ds
        Read Channel 00

```



Experiment Result – Timeline Trace

- u280 with DDR[1]
- image size:
32x32



Experiment Result – Profile Summary

- u280 with HBM[1]
- image size:
1080x640

[illegible]

Experiment Result – Profile Summary

- u280 with DDR[1]
- image size:
1080x640

[illegible]

Future Work

- Multiple DDR & HBM
- Multiple AXI channels (Max 4 in U280)